Agent Workflow

CSC3044S AI Systems

Lecturer: Dr Francois Meyer



Computer Science Department
University of Cape Town

LLM Agents Topic 4
9 October 2026

Based on [Huggingface Agents Course Unit 1]

How to build CLAUDE CODE from scratch

1. Scale the LLM pretraining phrase
2. Scale the LLM generation phase (test-time scaling)
3. Enable the LLM to use files and tools
4. **Wrap the LLM in an agentic framework**

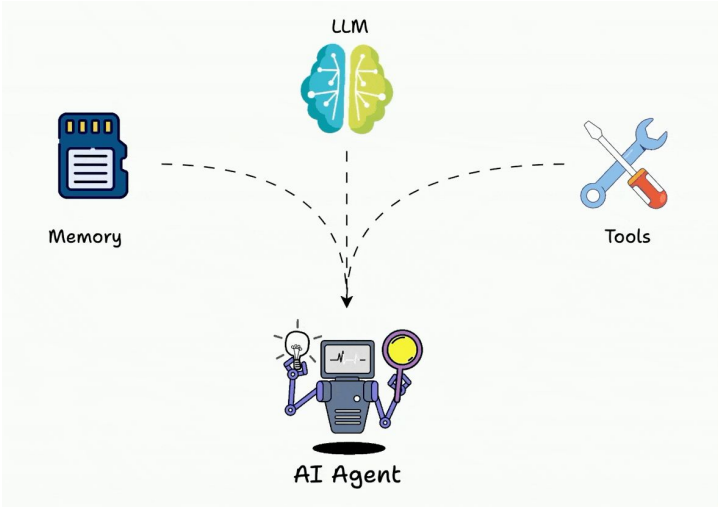
AI Agents

An agent is anything that can be viewed as perceiving its environment through sensors and acting upon that environment through actuators.

Russell & Norvig, AI: A Modern Approach

LLM Agent

An LLM agent is a system that leverages an LLM to interact with its environment to achieve a user-defined objective. It combines reasoning, planning, and the execution of actions (often via external tools) to fulfill tasks.



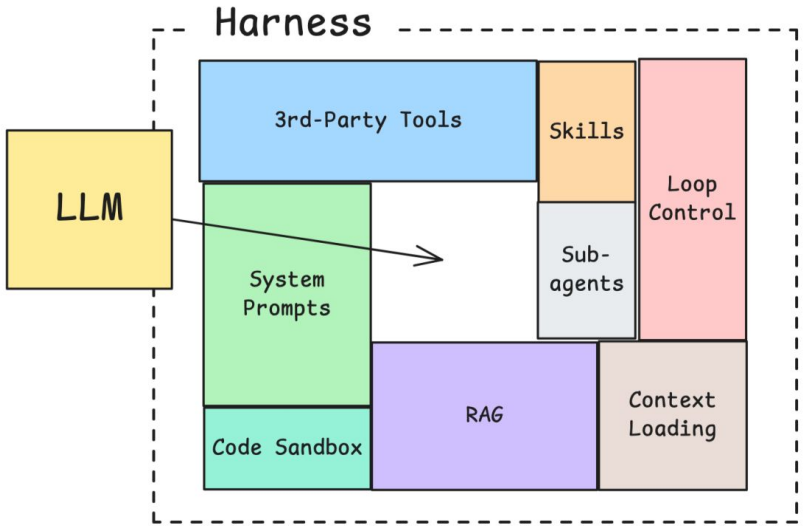
Harness

Agent harness is software infrastructure around an LLM that enables it to operate as an AI agent. It manages tool use, memory, state persistence, and feedback loops, as opposed to the model's internal reasoning.

model : harness

brain : body

CPU : OS



LLM Agent

LLM agents use LLMs as their core reasoning engine, to:

- **Understand natural language:** Interpret and respond to human instructions.
- **Reason and plan:** Analyze information, make decisions, and devise strategies to solve problems.
- **Interact with its environment:** Gather information, take actions, and observe the results of those actions.
  - **Environment:** codebase, browser, operating system.
  - **Observations:** file contents, tool results, codet output.
  - **Actions:** read a file, run a command, call an API, edit code.

LLM Agent Examples

Example 1: Coding Agents

Read codebase, write, edit files, run tests, and iterate until the task is complete.

Example 2: Customer Service Chatbots

Chatbot to answer questions, guide users, open issues in databases.

Example 3: Personal Virtual Assistants

Read your email and calendar, draft replies, book things.

Example 4: AI Non-Playable Character in a video game

Make NPCs more dynamic and able to interact with gamers contextually.

LLM Agent

What tasks can agents do? Any tasks performed via **tools** to complete **actions**.

E.g. If I have personal assistant agent and ask it to “send an email to my manager to delay today’s meeting”, I can give it code to send emails.

```
def send_email(recipient, message):

    # Send e-mail message to a recipient
```

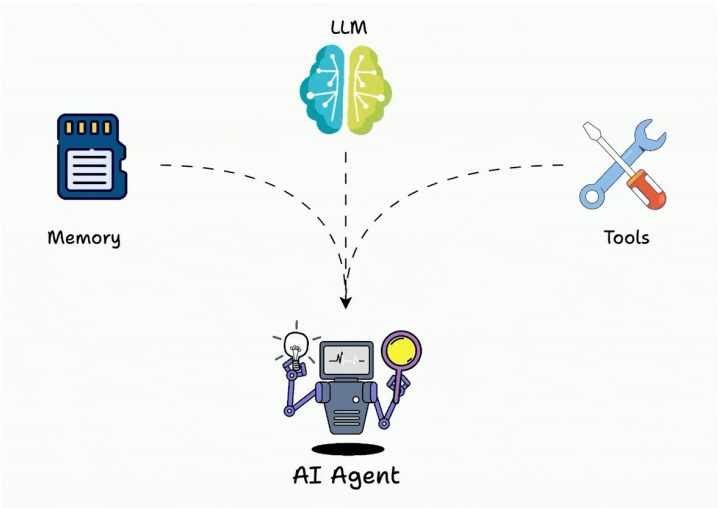
The LLM will generate a tool call to run the tool when it needs to:

```
send_email("Manager", "Can we postpone today's meeting?")
```

Tools

Tool design and interfacing will greatly impact the quality of your agent!

Some tasks require specific tools e.g. `send_email`, while others may be solved with general-purpose tools e.g. `web_search`.



LLM Agents

In the previous lecture we learned:

- **Test-time scaling:** how LLMs reason and plan.
- **Tools:** how to equip LLMs with tools.

How can we combine these components to maximise LLM capabilities?

With the complete AI Agent Workflow, a cycle we defined as **Thought-Action-Observation**. One example of this is **ReAct**.

ReAct

ReAct combines Reasoning (Thought) with Acting (Act).

It is a **prompting technique** that encourages the model to think step-by-step and interleave actions (like using tools) between reasoning steps.

Alternates between:

- **Thought:** internal reasoning
- **Action:** tool usage
- **Observation:** receiving tool output

ReAct Example

Thought: `I need to find the latest weather in Paris.`

Action: `Search["weather in Paris"]`

Observation: `It's 18°C and cloudy.`

Thought: `Now that I know the weather...`

Action: `Final answer: It's 18°C and cloudy in Paris.`

ReAct is a prompting technique. It is an example of a more general framework for agentic LLMs, which is called the Thought-Action-Observation Cycle.

Thought-Action-Observation Cycle

Thought-Action-Observation

Agents work in a continuous cycle of **thinking → acting → observing**.

- **Thought:** LLM part of the Agent decides what the next step should be.
- **Action:** agent performs an action, often by calling tools.
- **Observation:** model reflects on the response from the tool.

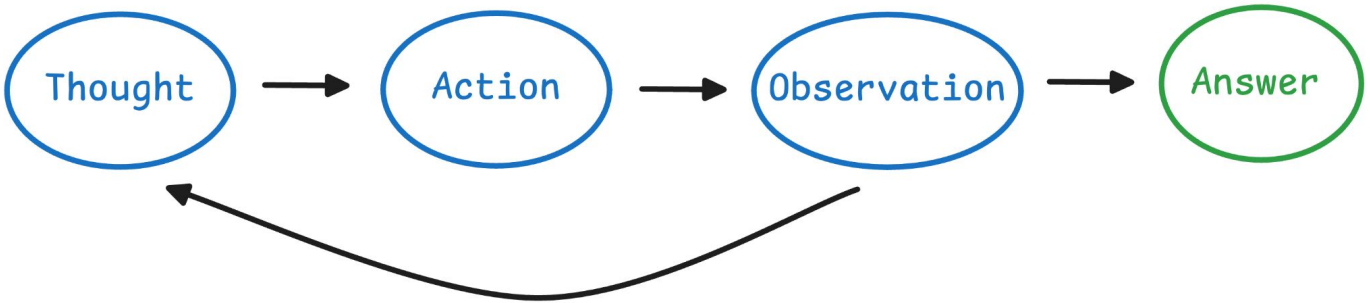


Image from https://www.dailydoseofds.com/ai-agents-crash-course-part-10-with-implementation/.

Thought-Action-Observation

These components work together in a continuous loop.

The loop continues until the objective of the agent has been fulfilled.

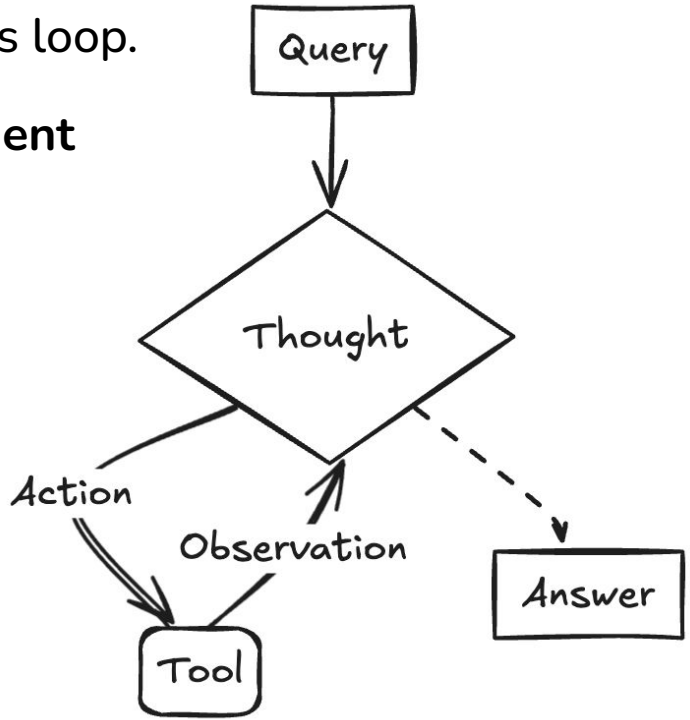


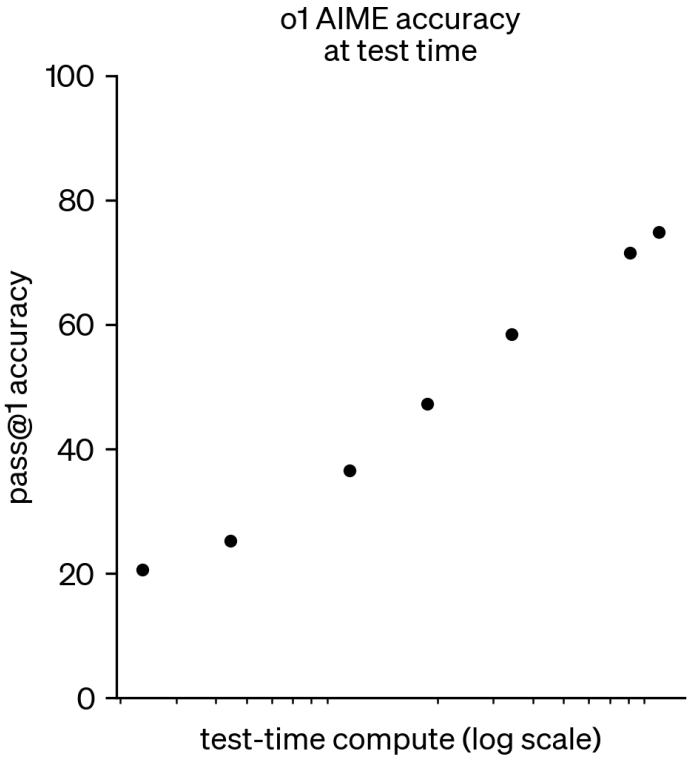
Image from https://github.com/philschmid/gemini-samples/blob/main/guides/langgraph-react-agent.ipynb.

Thought-Action-Observation

Thought:

- Utilises **LLM reasoning** capacity to reason about solving the task.
- **Goal:** assess current observations and decide on the next action(s).

Sophisticated forms of reasoning emerge from test-time scaling...



Type of Thought	Example
Planning	*I need to break this task into three steps: 1) gather data, 2) analyze trends, 3) generate report.*
Analysis	*Based on the error message, the issue appears to be with the database connection parameters.*
Decision Making	*Given the user's budget constraints, I should recommend the mid-tier option.*
Problem Solving	*To optimize this code, I should first profile it to identify bottlenecks.*
Memory Integration	*The user mentioned their preference for Python earlier, so I'll provide examples in Python.*
Self-Reflection	*My last approach didn't work well, I should try a different strategy.*
Goal Setting	*To complete this task, I need to first establish the acceptance criteria.*
Prioritization	*The security vulnerability should be addressed before adding new features.*

Thought-Action-Observation

Actions: The concrete steps an AI agent takes to interact with its environment.

The type of actions depend on the type of agent.

Type of Action	Description
Information Gathering	Performing web searches, querying databases, or retrieving documents.
Tool Usage	Making API calls, running calculations, writing/executing code.
Environment Interaction	Manipulating digital interfaces or controlling physical devices.
Communication	Engaging with users or collaborating with other agents.

Code Agents

Instead of calling special tools, write & execute code to perform the action.

Instruction: Determine the most cost-effective country to purchase the smartphone model "CodeAct 1". The countries to consider are the USA, Japan, Germany, and India.

Available APIs
[1] lookup_rates(country: str) -> (float, float)
[2] convert_and_tax(price: float, exchange_rate: float, tax_rate: float) -> float
[3] estimate_final_price(converted_price: float, shipping_cost: float) -> float
[4] lookup_phone_price(model: str, country: str) -> float
[5] estimate_shipping_cost(destination_country: str) -> float

LLM Agent using [Text/JSON] as Action

Think I should calculate the phone price in USD for each country, then find the most cost-effective country.

Action
```
Text: lookup_rates, Germany
JSON: {"tool": "lookup_rates", "country": "Germany"}
```

Environment 1.1, 0.19

Action
```
Text: lookup_phone_price, CodeAct 1, Germany
JSON: {"tool": "lookup_phone_price", "model": "CodeAct 1", "country": "Germany"}
```

Environment 700

Action
```
Text: convert_and_tax, 700, 1.1, 0.19
JSON: {"tool": "convert_and_tax", "price": 700, "exchange_rate": 1.1, "tax_rate": 0.19}
```

Environment 916.3

[... interactions omitted (look up shipping cost and calculate final price) ...]

Action
```
Text: lookup_rates, Japan
JSON: {"tool": "lookup_rates", "country": "Japan"}
```

[... interactions omitted (calculate final price for all other countries)...]

Response The most cost-effective country to purchase the smartphone model is Japan with price 904.00 in USD.

> **Fewer Actions Required!**

CodeAct: LLM Agent using [Code] as Action

Think I should calculate the phone price in USD for each country, then find the most cost-effective country.

Action

```
countries = ['USA', 'Japan', 'Germany', 'India']
final_prices = {}

for country in countries:
    exchange_rate, tax_rate = lookup_rates(country)
    local_price = lookup_phone_price("xAct 1", country)
    converted_price = convert_and_tax(
      local_price, exchange_rate, tax_rate
    )
    shipping_cost = estimate_shipping_cost(country)
    final_price = estimate_final_price(converted_price, shipping_cost)
    final_prices[country] = final_price

most_cost_effective_country = min(final_prices, key=final_prices.get)
most_cost_effective_price = final_prices[most_cost_effective_country]
print(most_cost_effective_country, most_cost_effective_price)
```

Control & Data Flow of Code Simplifies Complex Operations

Re-use `min` Function from Existing Software Infrastructures (Python library)

Environment 1.1, 0.19

Response The most cost-effective country to purchase the smartphone model is Japan with price 904.00 in USD.

Code Agents

Advantages:

- **Expressiveness:** Code provides greater flexibility than JSON.
- **Direct Integration:** Code Agents can integrate directly with external libraries, enabling more complex operations such as data processing.
- **Modularity and Reusability:** code is reusable across different actions

Disadvantages: Executing LLM-generated code may pose security risks!

Thought-Action-Observation

Observations: Agent perceives the consequences of its actions.

Signals from environment (e.g. API results, database changes, error messages, system logs) that guide the next cycle of thought and action.

Thought-Action-Observation

In the observation phase, the agent:

- **Collects feedback:** Receives data on whether action was successful or not.
- **Appends results:** Integrates the new information into its existing context, effectively ***updating its memory*** to adapt its strategy.

E.g. if a weather API returns the data “partly cloudy, 15°C, 60% humidity”, this observation is **appended at the end of the prompt before the LLM generates**.

Iterative feedback incorporation ensures agent remains dynamically aligned with its goals, constantly adjusting based on real-world outcomes.

Thought-Action-Observation

How is this cycle achieved for an LLM agent?

In many LLM agent frameworks, information about the AI Agent Workflow is provided in the **system prompt**, which is which is prepended to the user prompt.

This can include:

- Expectations about the Agent's behavior.
- Tools that the Agent has access to.
- Instructions on the Thought-Action-Observation Cycle.

Thought-Action-Observation

```
system_message="""You are an AI assistant designed to help users efficiently and accurately. Your
primary goal is to provide helpful, precise, and clear responses.

You have access to the following tools:
Tool Name: calculator, Description: Multiply two integers., Arguments: a: int, b: int, Outputs: int

You should think step by step in order to fulfill the objective with a reasoning divided into
Thought/Action/Observation steps that can be repeated multiple times if needed.

You should first reflect on the current situation using `Thought: {your_thoughts}`, then (if
necessary), call a tool with the proper JSON formatting `Action: {JSON_BLOB}`, or print your final
answer starting with the prefix `Final Answer:`
"""
```

Example

Weather agent: answer this query using a weather API tool.



Here's how the cycle unfolds...

Thought-Action-Observation

Thought (model reasoning):

Upon receiving the query, the LLM's reasoning trace might be...

```
The user needs current weather information for New York. I
have access to a tool that fetches weather data. First, I
need to call the weather API to get up-to-date details.
```

Thought-Action-Observation

Action (Tool Usage):

Based on its reasoning and the tools available to the LLM via the system prompt, the LLM generates a a JSON-formatted tool call:

```
I need to check the current weather for New York.

{"action": "get_weather",  "action_input": { "location":
"New York"}}
```

LLM stops generating! Harness parses the call and calls the tool.

Thought-Action-Observation

Observation (feedback from environment):

After the tool call, the harness receives an observation from the weather API:

```
Current weather in New York: partly cloudy, 15°C, 60% humidity.
```

This observation is then added to the prompt as additional context, the LLM is prompted to generate again, which allows it to update its internal reasoning.

***All of this is managed by the harness, which calls the tool, retrieves the response, appends it to LLM context, and prompts the LLM to generate again.**

Thought-Action-Observation

The LLM then reasons and generates a response:

```
I have the weather data now. The current weather in New York
is partly cloudy, 15°C and 60% humidity.

Final answer: The current weather in New York is partly
cloudy with a temperature of 15°C and 60% humidity.
```

The harness sees that the model has generated the final answer, which it extracts and shows to the user, closing the loop.

Thought-Action-Observation

How complex problems are solved:

- Agents **iterate** through this loop (potentially many times) until, during **Thought**, the model concludes it's ready the final answer.
- Each cycle incorporate new information (Observations) in its reasoning (Thought), taking it closer to the final answer.
- ***This behaviour can be learned with RL!***

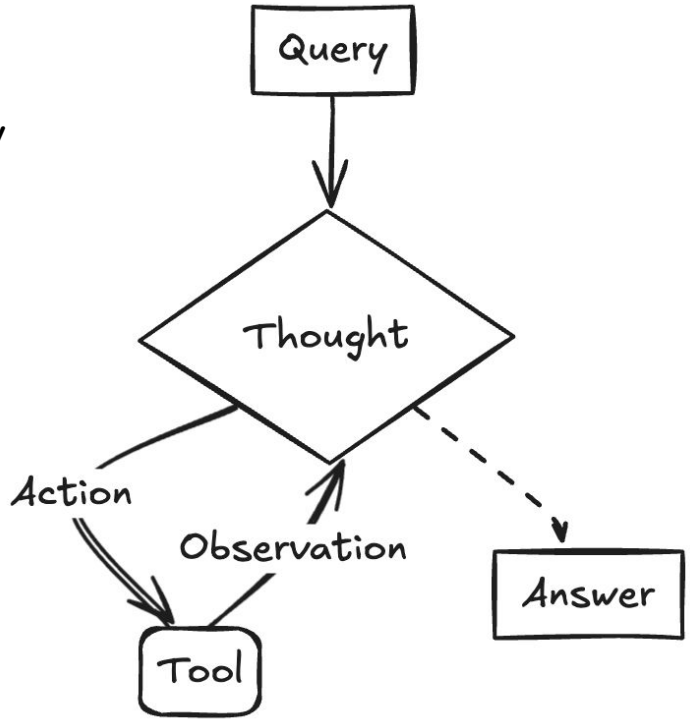
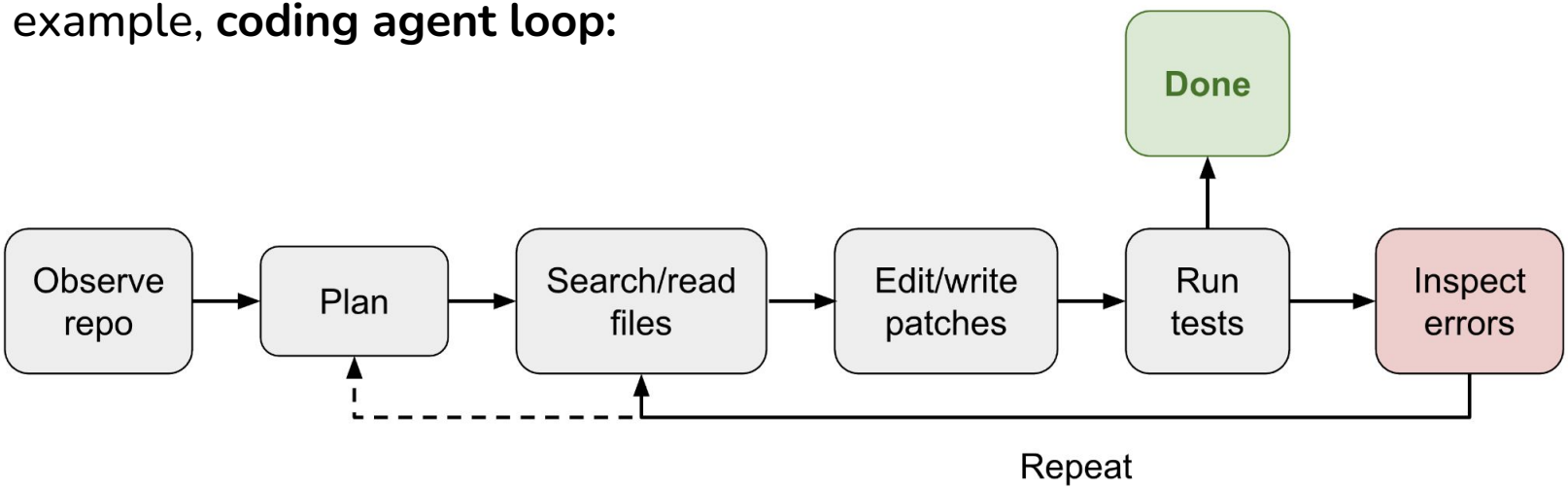


Image from https://github.com/philschmid/gemini-samples/blob/main/guides/langgraph-react-agent.ipynb.

Thought-Action-Observation

For different types of agents, this loop might look slightly different, but can be mapped to thoughts, actions, and observations.

For example, **coding agent loop:**



LLM Agents

LLM training: Change model behaviour through pretraining, supervised finetuning, or RL. Model acquires capabilities combining reasoning and tool use.

Harness engineering: Change the system around the model: prompts, tools, memory, and control flow.

LLM Agent capabilities emerge from the model-harness combination!

LLM Agents

“Eventually it is possible that many harness improvements will be internalized into core model behavior, but the interface with external context and tools should remain. We have seen a softer version of this pattern with prompt engineering: manual prompt tricks became less central as instruction tuning and model reasoning improved, but the need to specify goals, constraints, context, and evaluation did not disappear.”

What’s next for LLM agents? Bigger models? Better harnesses? Models that self-improve by designing harnesses for themselves?

Quote from Lilian Weng, “Harness Engineering for Self-Improvement” (lilianweng.github.io/posts/2026-07-04-harness)

Reading

- **These slides are the main resource to study for the exam.**
- **Prescribed reading:** Huggingface Agents Course Unit 1
  https://huggingface.co/learn/agents-course/en/unit1/introduction

How to build CLAUDE CODE from scratch

1. Scale pretraining (billions or trillions of parameters)
2. RLVR for test-time scaling
3. Equip with multiple tools (~20 according to Anthropic)
4. Wrap in an agentic loop and do some harness engineering!

